Charlie Flux

Registration number 100392431

2026

High Speed Communication Between Raspberry Pi via Secondary Memory Interface

Supervised by Gavin Cawley

University of East Anglia
Faculty of Science
School of Computing Sciences

Abstract

Acknowledgements

I would like to express my gratitude to my supervisor, Dr Gavin Cawley, for his support and guidance throughout the project and the engineers at Raspberry Pi for their insights and time.

Contents

1. Introduction **6**

2. Background **6**
2.1. Parallel Interfaces 6
2.2. User vs Kernel Space 7
2.2.1. Kernel Modules 8
2.2.2. Physical, Bus & Virtual Addressing 8
2.3. Hardware Interrupt Handling 9
2.3.1. Userspace I/O 9
2.4. Direct Memory Access (DMA) 9
2.5. Raspberry Pi Secondary Memory Interface (SMI) 10
2.5.1. SMI in Industry 10
2.5.2. SMI in Open-Source Projects 10
2.6. Dual Port SRAM 11
2.7. Application Programming Interface (API) Design 11
2.7.1. Defensive vs Contract API 12
2.7.2. Hardware APIs 12
2.8. Shared Memory Architecture 12

3. Methodology **13**
3.1. Work Process Overview 13
3.2. Software Environment 13
3.2.1. u-dma-buf 13
3.2.2. Build Configuration 14
3.3. Secondary Memory Interface Modes 15
3.4. System Design 15
3.5. Hardware Setup 15
3.6. Benchmark Environment 16

4. Implementation **16**
4.1. SMI Hardware 16
4.2. Software Implementation 17
4.3. SMI Kernel Module 17

4.4. SMI Driver 17
4.4.1. SMI Context Struct 18
4.4.2. Register Representation 18
4.4.3. Transfer Modes 19
4.4.4. Timeout Handling 19
4.4.5. DMA Transfers 20
4.4.6. Packing & Unpacking 20
4.5. API Design 21
4.5.1. API Implementation 21
4.6. SMI Lite 22
4.7. Shared Memory Implementation 22

5. Results and Evaluation **22**

6. Conclusion and Future Work **22**

References **22**

A. Figures **24**

B. Tables **24**

C. Gantt chart **30**

List of Figures

1. SMI Architecture Block Diagram 24
2. System Architecture 25
3. Software Architecture 26
4. 8-bit Interface Packing van Loo (2017) 27
5. Raspberry Pi 3B+ SMI Throughput Mb/s 27
6. Raspberry Pi 3B+ SMI Throughput MT/s 28
7. Raspberry Pi 4 SMI Throughput Mb/s 28
8. Raspberry Pi 4 SMI Throughput MT/s 29

List of Tables

1. SMI Direct Control and Status Register (DCS) 25

1. Introduction

Modern embedded systems offer versatile platforms for hobbyist and industrial applications with versatile peripheral interfaces and high-speed memory access. Among these platforms, Raspberry Pi established themselves as low-cost single board computer specialists, offering flexibility and accessibility for both research and industrial applications. However, hardware manufacturers moving away from parallel interfaces has silently deprecated support for legacy systems, forcing developers to rely on additional hardware or simulations to maintain compatibility.

This project focuses on the Secondary Memory Interface (SMI), produced by Broadcom and accessible on Raspberry Pi boards, designed to facilitate high-speed parallel memory access. Despite its unique nature, the SMI has seen limited adoption and has been described as an 'unloved peripheral' in industry practice Hughes (2026), contributing to the lack of widespread documentation and tooling. This project's aim is twofold: first, to provide comprehensive documentation and a user-space API for the SMI on Raspberry Pi; and second, to implement a functional shared memory system that demonstrates efficient data transfer between multiple processes and devices.

By combining thorough documentation and practical implementation, this project contributes both as a future reference for developers and as a practical example for high-speed shared memory usage on Raspberry Pi systems. The resulting implementation provides a platform for creating legacy support for deprecated parallel interfaces, while also encouraging further exploration of shared memory architectures on Raspberry Pi.

This paper presents the design, implementation and benchmarking of the Secondary Memory Interface on Raspberry Pi boards, highlighting performance differences across configurable modes and devices, in turn identifying best practices for efficient shared memory communication.

2. Background

2.1. Parallel Interfaces

The terms parallel and serial, when used to describe an interface, refers to the number of data lines used for communication. A parallel bus transmits multiple bits simultaneously across several lines whereas, a serial bus transmits data over a single line sequentially.

The popularity of parallel interfaces was driven largely by the weak compute power of the era, with data rates limited to megatransfers per second, widening the data bus was the only way to increase throughput. These interfaces were widely used for storage devices, PATA (IDE), and external printers via IEEE 1284 (often referred to as *Centronics* port).

Serial transmission offers clear advantages across the system, from silicon to software, due to advances in silicon technology and signal integrity allowing serial interfaces, such as USB and SATA, to achieve higher data rates. Modern implementations support multi gigatransfer per second data rates while minimising board area and lowers the risk of manufacturing defects. As a result, serial interfaces offer improved scalability and reduced hardware complexity compared to traditional parallel buses Hughes (2026). Parallel interfaces have been gradually deprecated, leaving legacy systems dependant on aging hardware or ad hoc solutions to maintain compatibility with modern storage and peripheral devices.

2.2. User vs Kernel Space

Modern operating systems implement memory and execution privileges as a hierarchical protection model, commonly referred to as privilege rings, which partitions execution into distinct levels of hardware access Tanenbaum (2015b). Linux makes use of two of these rings in practice: Ring 3 (user space) and Ring 0 (kernel space). Kernel space is reserved for the operating system itself with elevated hardware and system resources, whilst user space is where application code executes under controlled conditions with hardware access mediated by the kernel LDP (2026). The separation exists to maintain system integrity; by restricting direct access to hardware, a malicious user-space process cannot easily corrupt hardware state or destabilise the operating system. The kernel mediates elevated privileges through system calls, ensuring calls are validated before executing.

While the hierarchical protection model is a well-established method for enforcing integrity, the systems in place produce computational overhead in user space Minghao et al. (2007). Kernel space has been the de facto ring for implementation, offering streamlined performance and greater system control. This is a natural choice for hardware manufacturers, who can distribute drivers as part of official kernel builds with well-established security guarantees. However, for user-developed drivers, operating in

kernel space raises significant security implications, as providing unrestricted hardware access to an unverified source is heavily discouraged. Instead, user-developed drivers are more commonly implemented in user space, utilising existing kernel interfaces to provide support for functionality overlooked or undocumented by the vendor. Recent research argues that the overhead of user-space drivers are sufficiently low that the performance compromises become inconsequential compared to the increased safety Metzger et al. (2025). Furthermore, the degree of this overhead is largely a function of kernel design Liedtke et al. (1995); a sufficiently mature kernel will expose an efficient enough interface surface that user-space drivers become a fully viable alternative to kernel-space implementation.

2.2.1. Kernel Modules

Functionality is primarily loaded into the kernel at build time however, kernel modules allow functionality to be loaded and unloaded from the kernel on demand without rebooting the system, using tools such as `insmod` and `rmmod`. This flexibility makes kernel modules a practical platform for extending kernel functionality without modifying the kernel source. However, as kernel modules execute with full Ring 0 privileges, they carry the same security implications as built-in kernel code, making it desirable to keep any custom module as minimal as possible.

2.2.2. Physical, Bus & Virtual Addressing

Modern operating systems abstract physical memory from user and kernel space processes through virtual addressing. Each process operates within its own isolated virtual address space, with the Memory Management Unit (MMU) transparently translating virtual addresses to physical addresses at runtime. The isolation prevents processes directly accessing hardware peripherals or other processes' memory regions.

Physical addresses must be mapped into the virtual address space on Linux through `dev/mem`, which exposes the physical address space as a file that can be mapped using `mmap`, allowing peripheral registers to be accessed if they were ordinary memory.

The BCM283x introduces a third address space - bus addresses. Bus addresses are the addresses as seen from the perspective of the System on Chip interconnect fabric rather than the CPU. Whilst physical and bus addresses refer to the same underlying memory, the BCM283x VideoCore MMU applies a fixed offset between the two spaces, meaning

a physical and its corresponding bus address differ by a constant value. Bus addresses therefore represent the addresses that on-chip masters other than the CPU must use to correctly target a given memory address.

2.3. Hardware Interrupt Handling

A fundamental challenge of hardware driver design is responding to asynchronous hardware events without continuously polling the device, wasting processor cycles. Hardware interrupts address this by providing an asynchronous signal mechanism through which peripherals notify the processor directly when attention is required, temporarily suspending the current execution context to service the event. To manage the array of hardware interrupts that could be connected at any given moment, interrupt handlers are registered to specific interrupts to field each event and invoke the appropriate response. Hardware interrupts present a particular challenge for user-space driver development; while they are the primary mechanism by which hardware signals interact with software, interrupt handlers must be registered with the kernel and therefore cannot be implemented directly in user space.

2.3.1. Userspace I/O

To bridge this gap, the kernel can expose interrupt events to user space through a character device, allowing user-space processes to block on a `read` or `poll` clause until an interrupt occurs Koch (2006). This approach taken by the Linux *UIO* (Userspace I/O) subsystem, which provides a standard mechanism for exposing hardware interrupts to user space via a file descriptor, enables interrupt-driven design in user space. While the security implications of custom kernel modules are well understood, the simplicity of this mechanism makes a minimal interrupt-handling module a feasible and auditable supplement to a manufacturer's existing hardware support.

2.4. Direct Memory Access (DMA)

Naive memory transfers must be managed by the CPU, consuming processor cycles and preventing the concurrent execution of other tasks. In a system where multiple processes are active at the time of a transfer, the transfer speed will reduce due to resources being shared amongst the other processes. Rather than occupying the CPU for the duration of

the transfer, a DMA controller manages the movement of data independently, notifying the CPU via an interrupt upon completion. The decoupling of transfers from the CPU execution is particularly valuable in high throughput applications, where the cost of CPU-managed transfers would otherwise become the primary bottleneck.

2.5. Raspberry Pi Secondary Memory Interface (SMI)

The Secondary Memory Interface is a parallel interface implemented by Broadcom on the BCM283x family of system on chips (SoCs), commonly found on the Raspberry Pi product range. The SMI is intended as an external memory bus to interface with peripheral devices that would otherwise impose performance constraints if connected directly to the system bus van Loo (2017). It supports a range of register based devices and provides configurable bus widths along with precise timing control, making it well suited for interfacing with storage devices that operate at significantly lower clock speeds than the system bus.

2.5.1. SMI in Industry

Industry partners often receive extended documentation that is held under NDA, yet according to communication with Raspberry Pi engineers, this documentation contains little beyond what is publicly available Hughes (2026). Despite this limited additional coverage, industry implementations do exist; for instance, an official NAND flash driver Wren (2019) demonstrates the interface in practice and has been reported as the means by which firmware was loaded from a NAND flash chip on legacy Roku devices Dustin (2021).

2.5.2. SMI in Open-Source Projects

While industry implementations are seldom publicly documented, open-source examples do exist, most notably J Bentham's work on reverse engineering the SMI Bentham (2020). Bentham's work, while demonstrating that the SMI can be used effectively for high-speed parallel data transfer, is confined to specific ADC and DAC applications and operates at the direct register manipulation level with no abstraction layer or reusable API. This, combined with Bentham's self acknowledged lacunae surrounding the register definitions Bentham (2020), illustrates the current narrow state of documentation

available to developers.

A more complete, though still application specific, implementation can be found in the CaribouLite project Car (2026), an open-source SDR HAT for the Raspberry Pi. Rather than direct register manipulation, CaribouLite introduces *smi_stream_dev*, a custom kernel module that layers a cyclic DMA mechanism and software FIFO over the SMI hardware, exposing it as a standard character device. Despite this approach's greater sophistication, and its notably elegant polling mechanism, the implementation remains application specific and does not constitute a general purpose SMI API.

2.6. Dual Port SRAM

Static Random Access Memory (SRAM) is a high-speed volatile random access memory type. Unlike Dynamic Random Access Memory (DRAM), which stores data using capacitors, SRAM is a semiconductor based implementation using flip-flop logic to store data. Therefore, SRAM is favoured in smaller cache memory due to its high cost and high speed, whereas DRAM is primarily used for a system main memory.

The interface through which a memory device interacts is accessed is referred to as a port. Single-port SRAM permits only one read or write operation at any given time, which is sufficient for architectures with a single bus master. In a shared memory architecture arbitration logic is required to maintain concurrent access between multiple devices. Dual-port SRAM addresses this by providing two independent interfaces to a single address space, with integrated arbitration logic to handle simultaneous access, making it well suited to shared memory architectures despite its high cost.

2.7. Application Programming Interface (API) Design

An Application Programming Interface (API) defines the boundary between a software component and its users, specifying the operations available and valid inputs & outputs. While any functional interface technically constitutes as an API, the design of that interface has significant implications for usability, safety, maintainability and performance. A well designed API abstracts the complexity of the underlying system while exposing expansive functionality, allowing users to interact with hardware or software components without knowledge of the internal implementation.

2.7.1. Defensive vs Contract API

The two main philosophical approaches when designing an API are contract-based and defensive designs. Contract-based APIs make explicit promises about behaviour given valid inputs and places responsibility on the user to meet preconditions; resulting in a faster and leaner yet unforgiving API. A defensive API validates inputs and handles errors gracefully regardless of caller behaviour which requires a bulkier design due to additional input validation before a critical path is executed. For a hardware API the selection of philosophy is particularly consequential; an overly defensive design adds latency to every call, whilst an overly contract-based design leaves hardware state vulnerable to misuse Khlebnikov and Lakos (2018). The appropriate balance between these approaches in the context of SMI API is discussed in the implementation chapter.

2.7.2. Hardware APIs

When designing a hardware API, extra caution is required as mistakes can corrupt state in ways that are difficult to recover from, unlike software exceptions which can be caught and handled gracefully. Hardware APIs are also bound by timing constraints, meaning inefficient designs can consume valuable resources or violate the timing requirements of the underlying hardware, producing undefined behaviour.

2.8. Shared Memory Architecture

Shared memory is a communication paradigm in which multiple nodes share a single common shared physical memory space Hennessy et al. (2012). This design allows for fast data transfers and low-latency inter-process communication Tanenbaum (2015a) without the overhead of explicit message passing or serialisation. Unlike message passing, where data must be copied across address spaces and formatted for transmission, shared memory allows a producer to write directly to a region that a consumer can read immediately, minimising latency and maximising throughput.

Shared memory architectures can be realised at different levels of the system stack. At the software level, the operating system provides mechanisms, such as POSIX shared memory, allowing processes on the same machine to share common address spaces. At the hardware level, physically shared memory devices such as dual-port SRAM extend this concept to independent processes on separate buses, providing deterministic low-

latency communication without OS involvement.

A fundamental challenge of shared memory system is determining which system has logical ownership of a given address space at any point in time. Whilst hardware arbitration exists and handles bus level arbitration, preventing simultaneous electrical access to the same address, it provides no higher-level guarantee about the validity or readiness of the data it contains. A common solution is to reserve a region of the shared memory for ownership and flags or semaphores, which devices write to signal data is ready and ownership has transferred to the consuming device Tanenbaum (2015c). The specific design ownership mechanism adopted in this project is outlined in the methodology.

3. Methodology

3.1. Work Process Overview

This project's development process followed an iterative, prototype-driven approach, beginning with the software layer before introducing the hardware complexity. Given the limited and fragmented documentation surrounding the SMI, development began with a knowledge-building phase, consolidating available sources and reverse engineering register behaviour to establish an understanding of the interface. To build the documentation an initial prototype was built with supporting tooling, which served to validate that understanding in practice. Once functionality was validated, development progressed to the design and implementation of the SMI API, abstracting the low-level SMI behaviour into a reusable interface. The final stage integrated this software stack with the full hardware system, applying the API to the shared memory use case.

3.2. Software Environment

Although the SMI driver is designed to be standalone, not requiring third-party libraries, some external dependencies were used to address challenges outside the core scope of the project.

3.2.1. u-dma-buf

A fundamental challenge of user-space DMA is cache coherency; the CPU and DMA controller hold independent views of memory and without explicit coherency guaran-

tees, data written by one may not be visible to the other subsequently producing undefined behaviour. The CPU caches frequently accessed memory in L1/L2 cache to avoid slow main memory accesses whereas the DMA controller bypasses the cache entirely. When the CPU writes data, it may remain in the cache and not be flushed to main memory immediately hence the DMA controller may read stale data that the CPU has not flushed yet. Equally, the DMA controller may write data to the main memory which may not be visible to the CPU if it holds a cached copy of that address range. u-dma-buf Ichiro (2026) is a kernel module that allocates physically contiguous, cache-coherent memory regions and exposes them to user space via a device file, resolving this issue without requiring a custom kernel module.

The SMI driver is designed to accept a DMA buffer address directly, decoupling it from any specific allocation mechanism, a flexibility that aligns with the open-source nature of the project. Therefore, u-dma-buf is not a hard dependency; any cache-coherent physically contiguous buffer is sufficient. However, u-dma-buf was chosen as it provides the most straightforward means of obtaining such a buffer in user space and is used consistently throughout development for this reason. A fully featured DMA library was considered, but such libraries typically abstract buffer management in a way that conflicts with the driver's design of accepting a raw buffer address. Additionally, the SMI driver requires simple DMA controller functionality and would not take full advantage of an external DMA library. u-dma-buf was preferred as it resolves the allocation and coherency problem while remaining transparent to the driver.

3.2.2. Build Configuration

The driver is built using the GNU Compiler Collection (GCC) with a provided Makefile, compiled against the C11 standard with POSIX extensions enabled via `D_POSIX-_C_SOURCE=199309L`. The highest level of compiler optimisations were enabled via `-O3` and `-march=native`, allowing the compiler to take full advantage of the native instruction set of the target platform. All levels of optimisation were tested however, `O3` was chosen to remove any computational bottleneck and allow a full evaluation of the interface.

A set of driver-specific preprocessor directives are passed via `SMIFLAGS`. Most notably the Pi model flag (e.g. `-DPI3` or `-DPI4`) is a required build argument that determines the physical base address, which differs between Raspberry Pi models. Omitting

this flag will produce a compile-time error. Optional flags include `-DSMI_VERBOSE` to enable diagnostic logic and `-DSMI_COLOURFULL_ERRORS` to enable colour-coded error output, both of which are useful during development and debugging but can be omitted for production builds.

The Makefile separates source files into driver modules and application code, compiling each unit independently into a dedicated build directory before linking.

3.3. Secondary Memory Interface Modes

The Secondary Memory Interface (SMI) has a wide range of configurations, notably multiple bus widths, offering 8, 9, 16 and 18-bit modes, each with implications for throughput and hardware compatibility. Each mode was tested in isolation and evaluated against the constraints of the system, namely the interface width of the dual-port SRAM. An 8-bit bus width was chosen due to the byte-addressable SRAM used in the final design. The selected bus width additionally has implications for how data is represented in memory, which is addressed in the implementation chapter.

3.4. System Design

Figure 2 outlines the shared memory architecture implemented. The system contains two Raspberry Pi single board computers interfacing directly to a dual port SRAM shared memory via the SMI interface. To expand the address space available, dedicated loadable page address registers were also implemented to occupy the upper byte of the address with the lower 6 bits coming from the dedicated SMI address bus. As the page address registers are asynchronous to the SMI address bus, crossing a page boundary will add latency to read/write accesses during operation. Memory ownership is managed via a reserved flag region in the SRAM that signals data readiness and transfers logical ownership between devices.

3.5. Hardware Setup

Prototyping was completed on a breadboard due to their simplicity and ease of rapid prototyping. During testing issues with signal integrity on breadboards imposed constraints on the timing configuration however, the setup provided a proof of concept that could

be used for SMI debugging. These issues were addressed during the final implementation with all components being soldered to individual printed circuit boards (PCBs). Jumper wires were used to interconnect the PCB modules which maintained modularity and simplified debugging. The page address registers used Texas Instruments 74HC193 4-bit counters for their asynchronous incrementing and load operations.

3.6. Benchmark Environment

4. Implementation

The hardware implementation consisted of simple breakout boards for the SRAM and page address registers, with decoupling capacitors to stabilise the supply voltage. The primary implementation complexity therefore lies in the software layer.

4.1. SMI Hardware

Figure 1 illustrates the internal architecture of the SMI hardware. At its core, two asynchronous FIFOs (64×32-bit) bridge the AXI clock domain and the SMI clock domain, decoupling the system bus from a slower peripheral clock. Write data from the AXI slave is pushed into the transmit (TX) FIFO and then drained on to the SMI data bus, whilst incoming data from the SMI bus is captured into the receive (RX) FIFO and read back up via the AXI slave. The data format block handles the bus width configuration (8, 9, 16 and 18-bits) and the RX packing which allows received data to be packed into 16-bit RGB565 or 32-bit XRGB format furthermore, the data format block can swap the received word's bit order. External DMA requests may be passed through the top two bits of the data bus as paced by the DMA control block. The Transfer Timing Mode Select block generates external bus control signals such as the read and write strobe `SOE_N/SE` and `SWE_N/SRW_N`, which are physical signals used to trigger read and write operations on the connected peripheral. The interrupt control block monitors FIFO status and asserts relevant interrupt signal `o_smi_int` such as transfer completion or FIFO level threshold events. The 6-bit address bus `o_smi_addr` carries the device address to the external peripheral however, it can be repurposed as a general-purpose parallel output.

On the BCM283x, all peripherals are controlled through memory-mapped registers

residing at fixed physical addresses within the peripheral address space. An example register can be seen in Table 1.

The SMI can be programmed to carry out a sequence of transfers to a given address ahead of time (programmed transfer) or can be completed individually (direct transfer). Direct transfers are controlled via the direct control registers: Direct Control and Status `DCS`, Direct Address `DA` and Direct Data `DD`. Programmed transfers are controlled via the SMI registers: Control and Status `CS`, Length `L`, Address `A` and Data `D`. Programmed transfers can also be paced via DMA, controlled by the SMI DMA Control `DC` register. There are four SMI device settings that can be stored by Device Read Settings `DSR` and Device Write Settings `DSW` registers which are then selected at transfer configuration time in the `A` and `DA` registers.

4.2. Software Implementation

Figure 3 shows layered architecture of the software implementation, built from the ground up, exposing a general-purpose API to the end user. Each layer is designed to be as thin as possible, delegating complexity upwards and maintaining a clear separation of concerns. Whilst the top-level SMI API is intended for general use, the underlying layers are independently accessible, allowing users to bypass higher-level abstractions and build custom functionality directly on top of the lower layers if required. This modularity is a deliberate design principle, consistent with the open-source nature of the project. All code was written in C, consistent with the system programming requirements of the project and the language conventions of the Linux kernel ecosystem.

4.3. SMI Kernel Module

The SMI kernel module is an optional module that exposes the SMI's hardware interrupt to user-space via

4.4. SMI Driver

The SMI driver is the lowest level of implementation that interacts with the SMI registers. Functionality is split into modules, each accompanied by a matching header file, separating the public interface from the internal implementation. Whilst both the low-

level driver functionality and high-level API are implemented within `smi.c`, they are discussed separately to reflect their distinct roles in the software architecture.

4.4.1. SMI Context Struct

Rather than relying on global state, all register mappings, device configurations and buffer references are encapsulated within a context structure `SMI_CXT`, allowing multiple independent instances to coexist and making the driver's dependencies explicit at every call site. Context structs can be initialised and refreshed using helper functions such as `smi_init_cxt_map` to map peripheral registers and `smi_sync_context_device` to synchronise the device timing configuration with the current context state.

4.4.2. Register Representation

All registers can be represented as 32-bit unsigned integers however, raw integer manipulation requires complex bit-masking and shifting that obscures the intent of each operation. SMI registers are therefore represented using a union of a named bitfield struct and a raw `uint32_t`, allowing registers to be accessed by name whilst retaining the ability to read or write the entire register atomically.

Bitfield layout is not guaranteed by the C specification and is left to compiler discretion, making bitfield-based register representation non-portable in the general case. However, as the driver is specifically targeting BCM283x processors, the GNU Compiler Collection (GCC) is the de facto compiler for this platform. Under GCC on ARM, bitfield ordering is well defined and consistent, making this a safe and pragmatic design choice within the constraints of the system.

Each bitfield member is declared `volatile` to prevent the compiler from optimising away hardware register accesses, ensuring every read and write produces an actual memory transaction. The bitfield struct is declared with `__attribute__((packed))` to prevent the compiler inserting padding between members during optimisation, ensuring the in-memory layout corresponds exactly to the hardware register specifications. Furthermore, registers are aligned to 4-byte boundary using `__attribute__((aligned(4)))`, ensuring that register accesses are never split across word boundaries, which would produce undefined hardware behaviour on the BCM283x memory bus.

4.4.3. Transfer Modes

The driver exposes two transfer modes: direct transfers, which execute a single read or write operation immediately via the `DCS`, `DA` and `DD` registers, and programmed transfers, which configure the SMI to execute a sequence of transfers to a given address via the `CS`, `A`, `D` registers. Each mode is exposed through single and array variants, for example `smi_direct_read` and `smi_direct_read_arr`, providing a consistent interface regardless of transfer length. Each transfer mode is paired with a corresponding await function, such as `smi_read_await` and `smi_read_await_direct`, which block until the transfer completes.

4.4.4. Timeout Handling

To handle variable hardware latency without wasting processor cycles unnecessarily, await functions employ a tiered spin strategy. The system progressively trades responsiveness for CPU efficiency as the wait grows longer. This approach ensures high-performance, low-latency communication for fast operations while minimising CPU utilisation during extended stalls. The waiting logic transitions through four distinct tiers based on the incrementing spin counter.

- **Tier 0 (Active Polling):** A phase designed for minimal latencies. By continuously polling the hardware state, the CPU pipeline remains hot, ensuring the lowest possible latency upon state transition.
- **Tier 1 (Execution Hint):** The insertion of a `NOP` instruction with a compiler memory barrier prevents load/store reordering around the spin loop without surrendering the thread's scheduler quantum.
- **Tier 2 (Process Yielding):** Utilisation of the `sched_yield` system call. This instructs the OS scheduler to move the thread to the end of the run queue for its current priority level, allowing other tasks to execute while remaining runnable.
- **Tier 3 (Blocked Sleep):** For extended delays, the driver invokes the POSIX `nanosleep` interface (defined in `time.h`). This blocks the calling thread, fully releasing the CPU core to the kernel until the specified interval elapses.

The thresholds governing tier transitions are compile-time configurable via preprocessor definitions, allowing the spin behaviour to be tuned for different hardware characteristics without modifying the driver logic. The tier transitions were determined empirically by profiling transfer completion spin counts across a range of operating conditions. Measurements were taken both idle and loaded system conditions to characterise the boundary between hardware latency and OS scheduling jitter.

4.4.5. DMA Transfers

Without DMA, the CPU must manage all data transfers directly, polling the SMI FIFO and writing or reading word by word. This is a resource-intensive operation that competes with other processes on the scheduler. Offloading transfers to the dedicated to the dedicated DMA controller frees the CPU during the transfer, which is particularly beneficial under a busy scheduler.

A DMA transfer is described via a Control Block specifying the source address, destination address and transfer length, along with pacing configuration. The SMI is enabled for DMA operation via the `DC` register, which configures the DREQ threshold for both read and write directions. A cache-coherent buffer is allocated via u-dma-buf and tracked within `SMI_CXT`, providing a memory region accessible by both the CPU and DMA controller without coherency issues. For write transfers, the Control Block source is set to the buffer's bus address and the decision to the bus address of the SMI `D` register. For read transfers these are inverted, with the `D` register as the source and the cache-coherent buffer as the destination.

4.4.6. Packing & Unpacking

The SMI FIFO interfaces with the AXI bus which operates on 32-bit words regardless of the configured SMI bus width. As a result, all data passing through the FIFO is padded to 32-bit width, meaning an 8-bit transfer consumes a full 32-bit FIFO entry. To mitigate this inefficiency, the SMI hardware supports optional packing modes which consolidate multiple bus-width samples into a single 32-bit FIFO entry. Data may be packed into either 16-bit RGB565 or 32-bit XRGB format, as illustrated in Figure 4, reducing the number of memory transfers required by up to 4$\times$ in the case of 8-bit RGB565 packing and significantly improving transfer throughput.

When packing is not enabled, received words are truncated to the configured bus

width via the `smi_truncate` family of functions, discarding the unused upper bits of each 32-bit word. When packing is enabled, the more complex unpacking process is required to reconstruct the original samples from the consolidated FIFO words.

Unpacking function selection is managed through the `SMI_CXT` context, which dispatches the appropriate unpacking function based on the configured bus width and pixel format. Each bus width and format combination has a dedicated unpacking function, with performance-critical word-level operations implemented as `static inline` functions to eliminate function call overhead in the hot path. Lookup tables were considered for unpacking but disregarded due to the limited L1 cache available on BCM283x processors; cache misses on a lookup table would incur greater latency than the equivalent branch instructions, justifying the trade-off of code readability for performance. The SMI also has the ability to swap the bit order FIFO words to support little-endian peripherals. Dedicated swap variants are provided for each unpacking function to handle this configuration.

4.5. API Design

The API layer is built directly on top of the SMI driver, exposing a general-purpose interface for SMI communication. A deliberate distinction is made between the public-facing API and internal worker functions in terms of input validation and philosophy.

4.5.1. API Implementation

Both defensive and contract-based design present trade-offs that are particularly consequential for a hardware API. The SMI API adopts a hybrid approach, applying each philosophy where it is most appropriate within the software architecture. Public-facing interfaces such as `smi_direct_read` and `smi_programmed_read` adopt defensive design, validating inputs and returning error codes on failure to ensure safe usage at the API boundary. Internal worker functions, including the await and unpacking routines, adopt contract-based design, assuming valid inputs as a precondition and eliminating redundant validation from performance-critical paths. The hybrid approach ensures safety at the public interface without compromising the performance of internal transfer and unpacking operations. Furthermore, the open-source nature of the driver encourages users to build their own abstractions directly on top of the lower-level interfaces, leveraging the contract-based internals where performance is a priority.

4.6. SMI Lite

`smilite.h` is a single-header implementation of the SMI driver designed for users who require a lightweight integration path without the overhead of the fill multi-file driver. Single-header libraries are a common pattern in C development, allowing entire implementation to be included into a project by adding a single `#include` directive, eliminating the need for a separate compilation step or build system integration.

`smilite.h` includes the complete register definitions, memory mappings utilities, address translation macros, timeout handle, await routines and the full suite of unpacking and truncation functions. However, it deliberately omits the higher-level initialisation helpers such as `smi_init_cxt_map` and `smi_init_rw_config`, leaving peripheral setup and context initialisation to the user. The user must also define system specific macros, most notably `PHYS_REG_BASE` which specifies the physical register address base which differs across BCM283x variants, enforced at compile time via a preprocessor error. These requirements makes the platform dependencies explicit rather than silently assuming a specific hardware configuration, improving portability across BCM283x variants.

4.7. Shared Memory Implementation

5. Results and Evaluation

6. Conclusion and Future Work

References

(2026). Cariboulabs/cariboulite. CaribouLabs.co.

Bentham, J. (2020). Raspberry Pi Secondary Memory Interface (SMI).

Dustin (2021). Raspberry Pi Hat Adds SDR With High Speed Memory Access.

Hennessy, J. L., Patterson, D. A., and Asanović, K. (2012). Shared Memory Architecture. In *Computer Architecture: A Quantitative Approach*, pages 351–352. Morgan Kaufmann/Elsevier, Waltham, MA, 5th ed edition.

Hughes, J. (2026). Raspberry Pi Engineer Communications.

Ichiro, KAWAZOME. (2026). Ikwzm/udmabuf.

Khlebnikov, R. and Lakos, J. (2018). Contracts, Undefined Behavior, and Defensive Programming.

Koch, H.-J. (2006). The Userspace I/O - The Linux Kernel documentation. https://www.kernel.org/doc/html/v4.14/driver-api/uio-howto.html.

LDP (2026). Introduction — The Linux Kernel documentation. https://linux-kernel-labs.github.io/refs/heads/master/lectures/intro.html.

Liedtke, J., Hartig, H., Hohmuth, M., Schonberg, S., and Wolter, J. (1995). The Performance of μ-Kernel-Based Systems.

Metzger, P., Markettos, A. T., Napierała, E. T., Naylor, M., Watson, R. N. M., and Jones, T. M. (2025). Deprivileging Low-Level GPU Drivers Efficiently with User-Space Processes and CHERI Compartments.

Minghao, K., Chyang, K. Y., and Karuppiah, E. K. (2007). Performance Analysis and Optimization of User Space versus Kernel Space Network Application. In *2007 5th Student Conference on Research and Development*.

Tanenbaum, A. S. (2015a). Interprocess Communication. In *Modern Operating Systems*, pages 119–120. Pearson, Boston, fourth edition edition.

Tanenbaum, A. S. (2015b). Layered Operating System Model. In *Modern Operating Systems*, pages 62–65. Pearson, Boston, fourth edition edition.

Tanenbaum, A. S. (2015c). Semaphores and Mutexes. In *Modern Operating Systems*, pages 130–137. Pearson, Boston, fourth edition edition.

van Loo, G. (2017). Secondary Memory Interface (SMI).

Wren, L. (2019). NAND flash driver for Broadcom Secondary Memory Interface. https://github.com/raspberrypi/linux/blob/e2d2941326922b63d722ebc46520c3a2287b675f/drivers/mtd/

A. Figures

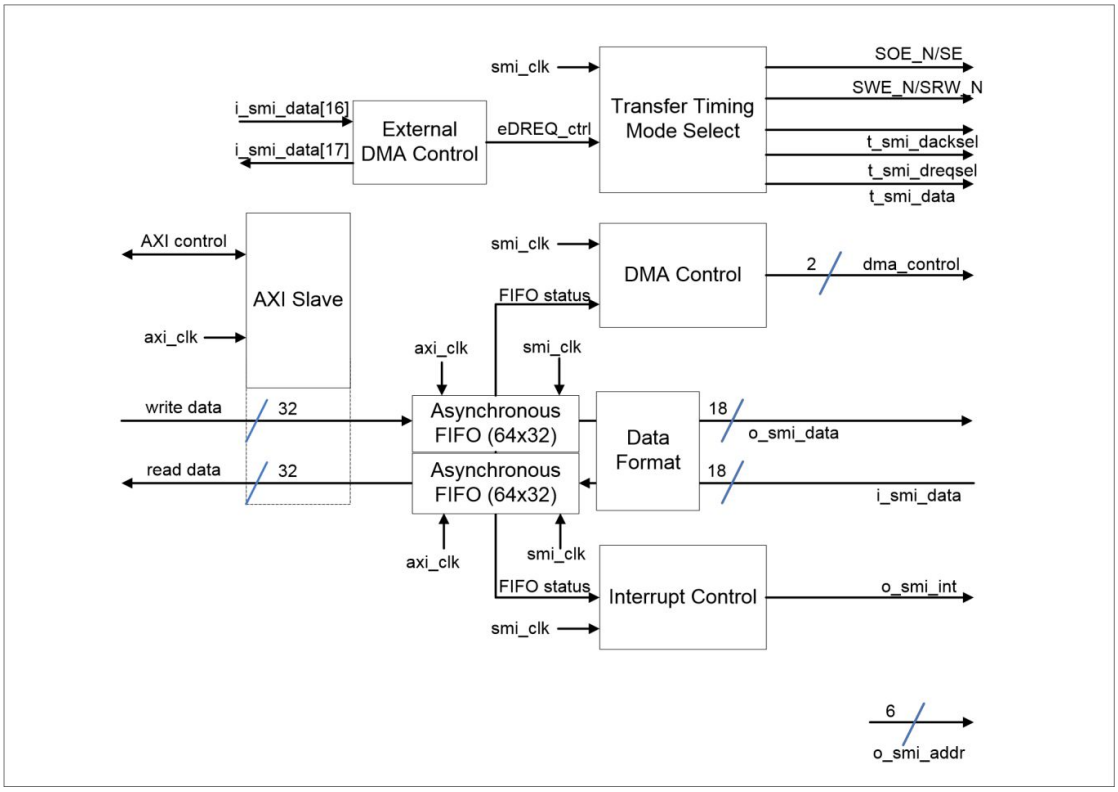

Figure 1: SMI Architecture Block Diagram

B. Tables

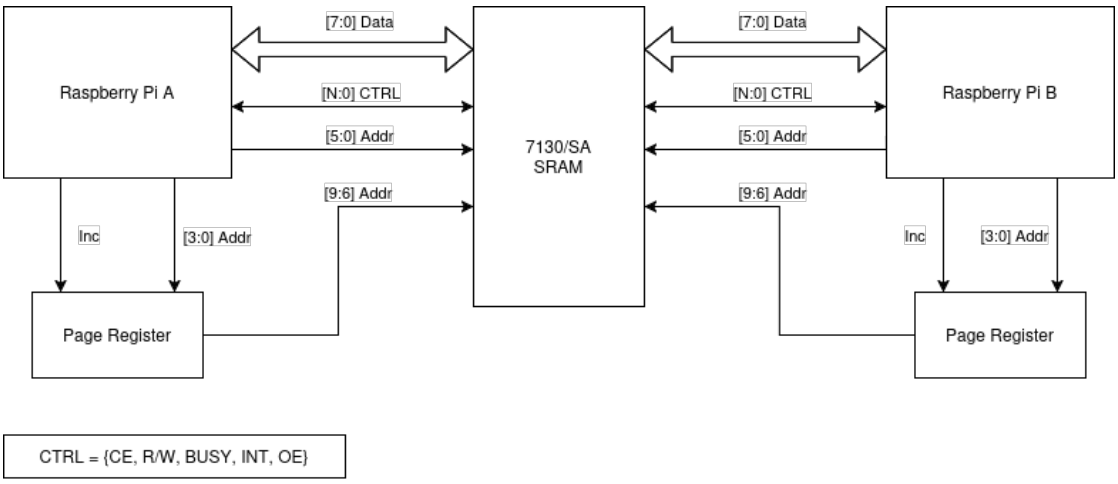

Figure 2: System Architecture

Bit(s)	Field	Description	Type	Reset
31–4	—	Unused	-	-
3	WRITE	Transfer Direction. 0 = Read from external device. 1 = Write to external device.	R/W	0
2	DONE	Direct Transfer Complete. 0 = No meaning. 1 = Transfer has finished. Writing 1 clears this flag.	R/W	0
1	START	Start Transfer. Writing 1 starts a transfer if one is not already in progress.	W	0
0	ENABLE	Enable the SMI. OR'd with the ENABLE bit in SMI_CS. 0 = Disable (power-saving state). 1 = Enable.	R/W	0

Table 1: SMI Direct Control and Status Register (DCS)

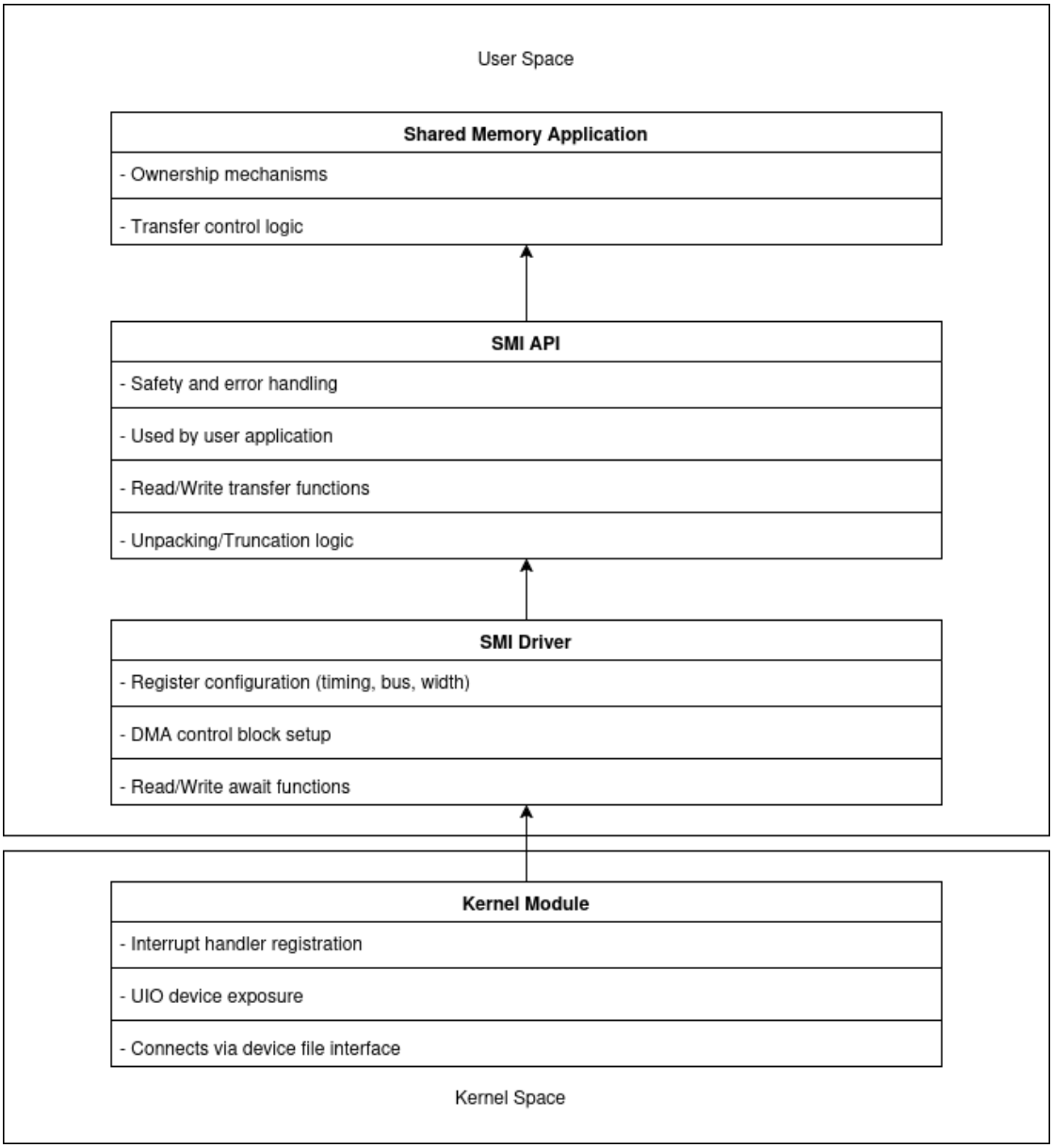

Figure 3: Software Architecture

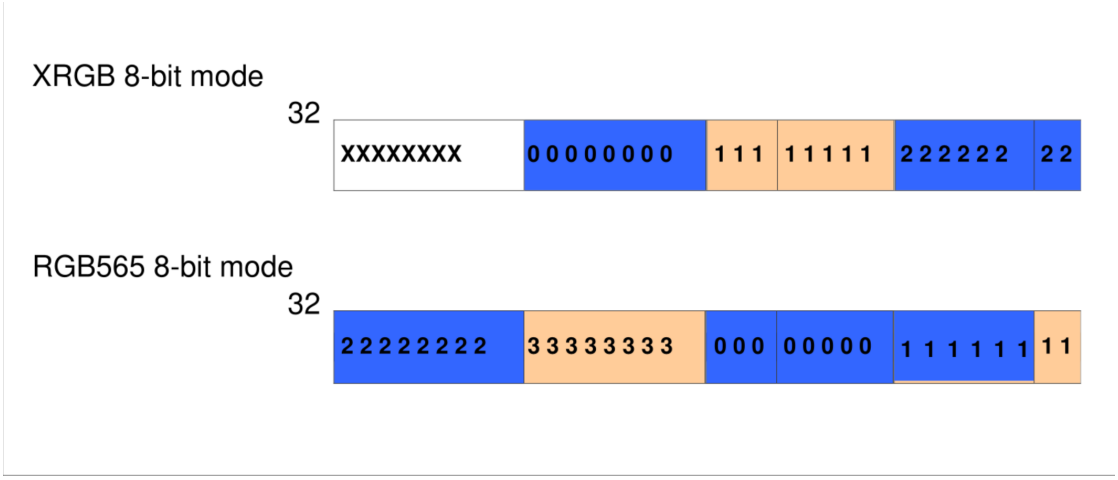

Figure 4: 8-bit Interface Packing van Loo (2017)

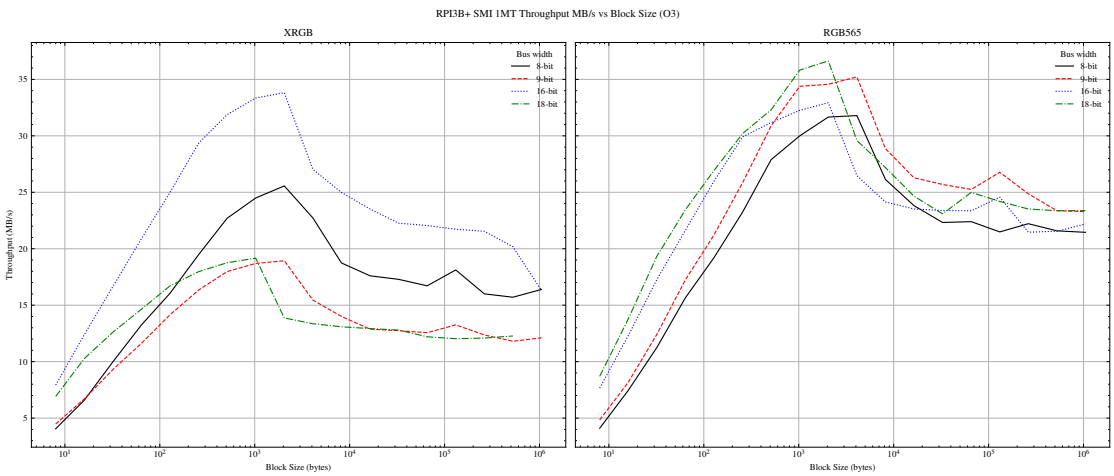

Figure 5: Raspberry Pi 3B+ SMI Throughput Mb/s

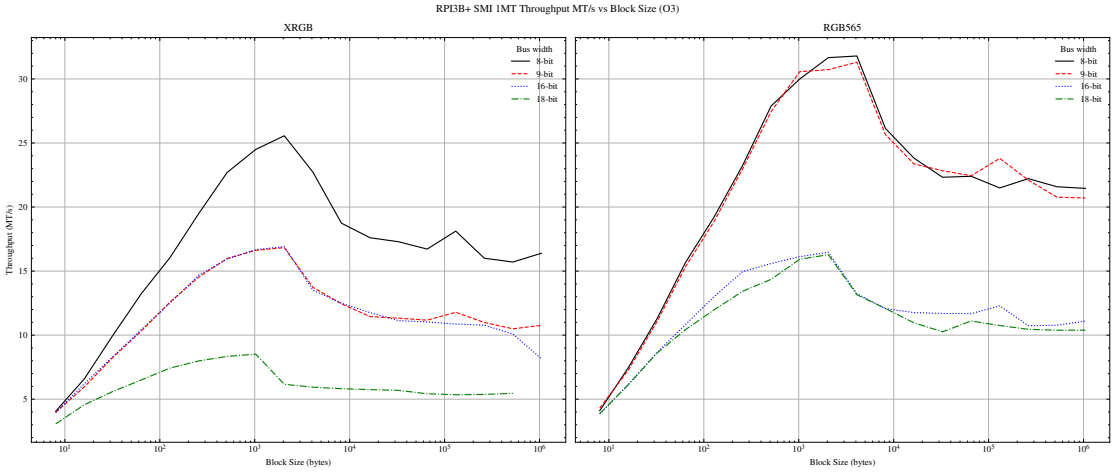

Figure 6: Raspberry Pi 3B+ SMI Throughput MT/s

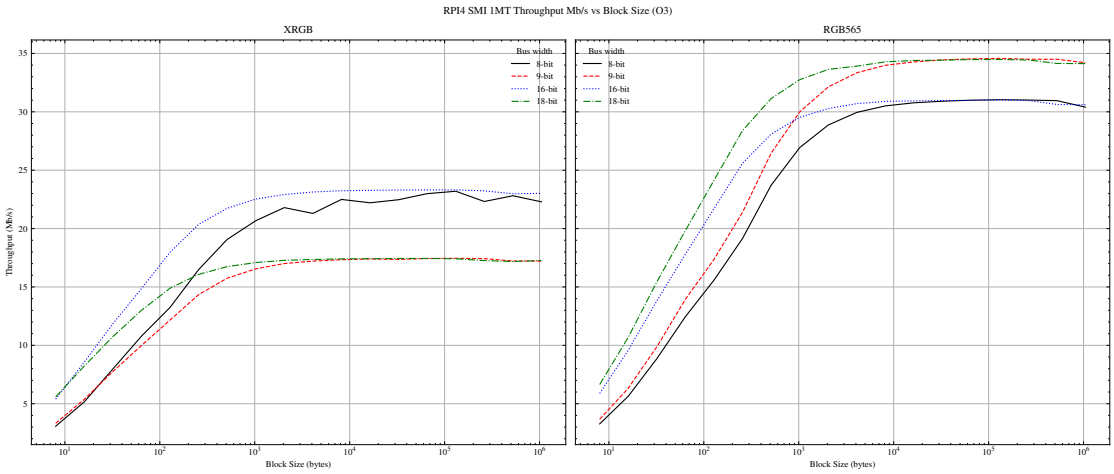

Figure 7: Raspberry Pi 4 SMI Throughput Mb/s

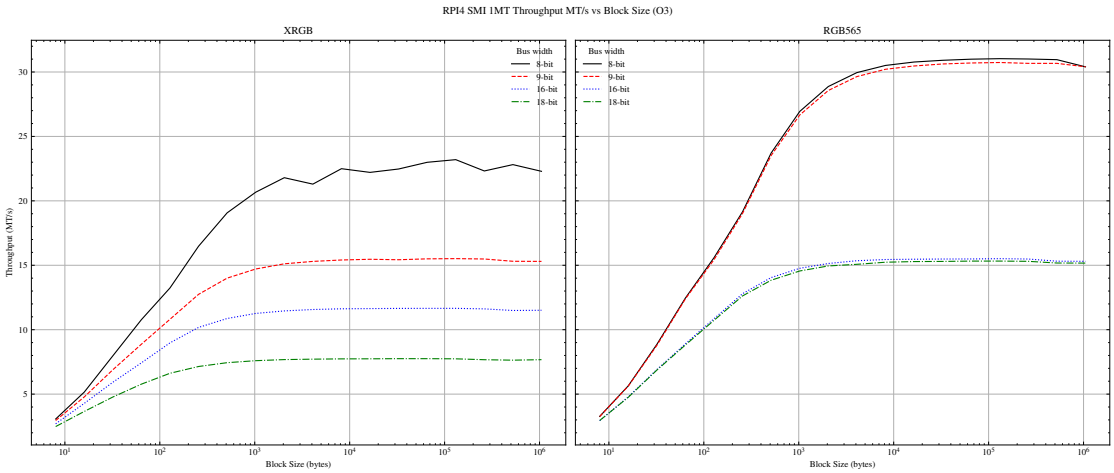

Figure 8: Raspberry Pi 4 SMI Throughput MT/s

C. Gantt chart